

Relatório técnico

Projeto Integrador HemoTrack / Rota Vital

Demanda preditiva com threads

US07 | Análise Preditiva de Demanda por Tipo Sanguíneo

Serviço Spring Boot sequencial, com threads de plataforma e com threads virtuais do Java 21: implementação, medições e análise.

Infraestrutura de Software

2026.2 | ADS

sequencial

4 threads



Sumário

01	Apresentação	3
02	A história escolhida: US07	4
03	Por que a US07: justificativa técnica	5
04	Arquitetura da solução	7
05	Estrutura do projeto	8
06	O código, classe por classe	9
07	Endpoints da API	18
08	A interface de demonstração	19
09	Como executar e medir	21
10	Resultados de negócio: a projeção da US07	23
11	Medições de desempenho	25
12	Análise dos resultados	27
13	Opcional: threads virtuais do Java 21	28
14	Race condition: por que acumuladores locais	30
15	Como a arquitetura pode evoluir	31
16	Conclusão	31
17	Referências	32

01

Apresentação

O que a atividade pediu, o que o grupo escolheu e o que foi entregue.

O HemoTrack (Rota Vital) nasce como um sistema web em três camadas: apresentação, aplicação (API) e banco de dados. A atividade parte de uma premissa de escala nacional: milhares de hospitais, milhões de requisições registradas e painéis consultados o dia inteiro. Nessa escala, alguma operação da camada de aplicação passa a demorar demais em uma única thread. O objetivo foi **encontrar essa operação, implementá-la como um serviço real e acelerá-la com threads**, medindo e explicando o ganho.

A escolha do grupo

Escolhemos a **US07, Análise Preditiva de Demanda por Tipo Sanguíneo**. Entre as sete histórias do HemoTrack, é a única que, na escala nacional, gasta o tempo **calculando** sobre milhões de registros e cujos dados podem ser divididos em fatias independentes. A justificativa completa está na seção 3 e no documento de justificativa entregue à parte.

1.1 O que a atividade pede e onde cada item está neste relatório

Item pedido	Como atendemos	Seção
Justificativa: operação, Big-O, gargalo, particionamento	Triagem das 7 histórias, análise de complexidade, gargalo de CPU e argumento de particionamento	3
Serviço real em Java com Spring Boot	Endpoint <code>GET /api/demanda/preditiva</code> , com geração de 100 mil a 10 milhões de registros	4 a 8
Duas versões: sequencial e com threads (2, 4, 8)	<code>ExecutorService</code> , uma fatia por thread, acumulador local e agregação ao final	6
Mesma resposta nas duas versões	Assinatura SHA-256 do resultado, testes JUnit e endpoint de verificação	6, 13
Medições: tabela, gráfico e speedup	Bateria automática na interface (mediana, aquecimento do JIT) e exportação CSV/Markdown	10, 11
Análise (linearidade, Big-O, concorrência × paralelismo, evolução)	Discussão com Lei de Amdahl e com os números medidos	12
Opcional: virtual threads do Java 21	Modo <code>virtual</code> no mesmo endpoint, medido lado a lado	13

1.2 O que foi entregue

- **Projeto Spring Boot** (`hemotrack-demanda`), Java 21, com o algoritmo em um pacote independente do Spring (`core`) e as camadas `service` e `api`.
- **Quatro modos de execução** no mesmo endpoint: sequencial, threads de plataforma, threads virtuais e uma versão insegura, errada de propósito, para demonstrar race condition.
- **Interface web de demonstração** servida pelo próprio Spring, com a linha do tempo animada das threads, a bateria de medições, os gráficos e a projeção por tipo sanguíneo.

- **Testes automatizados** que garantem que todas as versões devolvem exatamente o mesmo resultado.
- **Documentação**: justificativa, análise e este relatório.

02

A história escolhida: US07

Análise Preditiva de Demanda por Tipo Sanguíneo, e como cada critério de aceite foi atendido.

US07 | Análise Preditiva de Demanda por Tipo Sanguíneo

História: como analista de dados do sistema, eu gostaria de visualizar as projeções de demanda futura e os desvios estatísticos de consumo, para que as campanhas de doação sejam direcionadas aos tipos sanguíneos com maior risco de desabastecimento.

Cenário: projeção de demanda calculada com parâmetros estatísticos

Dado que a base histórica de consumo dos últimos meses foi processada

Quando o analista acessa o módulo de "Demanda Preditiva" para o próximo trimestre

Então o sistema plota a média estimada (μ) e as faixas de dispersão (σ) para cada tipo sanguíneo (A, B, AB, O, Rh+/-)

E destaca os tipos com probabilidade de ruptura de estoque (ex.: O-).

A história tem valor claro para a operação de um hemocentro: saber com antecedência quais tipos vão faltar permite antecipar campanhas de doação em vez de reagir ao desabastecimento. Para o objetivo técnico da atividade, ela tem uma característica decisiva: para responder à pergunta do analista, o sistema precisa **varrer todo o histórico nacional de requisições**, e esse histórico cresce todos os dias.

2.1 Critério de aceite × implementação

Critério	Como o sistema cumpre
Dado que a base histórica foi processada	A base de requisições das últimas 104 semanas epidemiológicas é carregada em memória (<code>POST /api/dados</code> , 100 mil a 10 milhões de registros). Cada chamada do endpoint processa essa base inteira.
Quando o analista acessa o módulo para o próximo trimestre	<code>GET /api/demanda/preditiva</code> projeta as 13 semanas seguintes (um trimestre). Na interface, o botão "Comparar" dispara a análise.
Então plota μ e as faixas σ por tipo	A resposta traz, para os 8 tipos (A+, A-, B+, B-, AB+, AB-, O+, O-), a média semanal μ , o desvio σ , a tendência e a projeção $\pm \sigma\sqrt{13}$. A interface plota a faixa $\mu \pm \sigma$ de cada tipo (Figuras 3 e 4).
E destaca os tipos com risco de ruptura	Cada tipo recebe P(ruptura) e um nível de risco (baixo, moderado, alto). A lista <code>tiposEmAlerta</code> e o destaque visual mostram O- em risco alto (100%) e AB- em risco moderado (21,9%).

2.2 O que o sistema calcula, em linguagem de negócio

- **Consumo semanal por tipo:** quantas bolsas de cada tipo foram requisitadas em cada semana, considerando o fuso horário de cada hospital.
- **Média (μ) e dispersão (σ):** o consumo típico por semana e o quanto ele oscila.
- **Tendência:** se o consumo está subindo ou caindo (regressão linear sobre as 104 semanas). O 0- cresce cerca de 31,6 bolsas por semana.
- **Projeção do trimestre:** a demanda esperada nas próximas 13 semanas, com margem de incerteza.
- **Probabilidade de ruptura:** a chance de a demanda projetada superar o estoque atual.

03

Por que a US07: justificativa técnica

Operação escolhida, Big-O, onde está o gargalo e por que os dados são particionáveis.

Seguimos o roteiro da atividade: listamos as operações de cada história e, para cada uma, perguntamos se na escala nacional o tempo é gasto **calculando** ou **esperando** (banco, rede), qual a Big-O e se os dados podem ser divididos em fatias independentes. Só serve a operação cujo gargalo é processamento e que dá para particionar; se a resposta certa for "melhorar a consulta SQL", ela não serve.

História	Onde o tempo é gasto	Big-O	Particionável?	Veredito
US01 Requisição emergencial	banco (1 INSERT)	$O(1)$	não há volume	não serve
US02 Entrada de bolsas	banco (1 INSERT)	$O(1)$	não há volume	não serve
US03 Alocação FEFO	consulta com índice (tipo, validade)	$O(\log n)$	não: pedidos disputam o mesmo estoque	não serve
US04 Rota (Dijkstra)	CPU, mas grafo pequeno por pedido	$O((V+E) \log V)$	só em lote de muitas rotas	candidata fraca
US05 Telemetria	rede, eventos simultâneos	$O(1)$ por leitura	é concorrência, não volume	não serve
US06 Painel gerencial	contagens e médias	$O(n)$ no banco	sim	um GROUP BY resolve
US07 Demanda preditiva	CPU, milhões de registros	$O(n)$	sim	escolhida

3.1 Big-O da solução sequencial

Sejam n o número de requisições no histórico, $t = 8$ os tipos sanguíneos e $w = 104$ as semanas.

```
percorrer o histórico e somar por tipo × semana .....  $O(n)$   
calcular  $\mu$ ,  $\sigma$ , tendência e projeção nas 8 séries .....  $O(t \cdot w) = O(832)$ , constante  
total sequencial .....  $O(n + t \cdot w) = O(n)$   
memória extra .....  $O(t \cdot w)$ 
```

O custo medido fica em torno de **49 ns por registro** e se mantém constante de 1 a 10 milhões de registros (Figura 9): dobrar a base dobra o tempo.

3.2 Onde está o gargalo

O painel preditivo é consultado o dia todo e o analista muda período e parâmetros. Por isso a base é lida do banco uma vez para um **snapshot em memória** na camada de aplicação, e cada chamada trabalha sobre esse snapshot. O tempo do endpoint, então, não tem espera de banco: é laço sobre os registros. O trabalho por registro é cálculo (converter o instante UTC para a data local do hospital, achar a semana epidemiológica e acumular). E não é caso de "melhorar o SQL": mesmo com a melhor consulta, continua sobrando $O(n)$ de cálculo na aplicação, e o banco transacional está ocupado com o caminho crítico (US01, US02 e US05).

3.3 Por que os dados são particionáveis

- **Registros independentes:** cada requisição contribui para uma única célula tipo × semana.
- **Soma de inteiros é associativa e comutativa:** somar fatias separadas e depois os parciais dá o mesmo total, em qualquer ordem.
- **Leitura compartilhada, escrita local:** a lista é somente leitura e cada thread escreve só no seu acumulador. Sem escrita compartilhada, sem race condition e sem trava.
- **Redução barata:** juntar os parciais custa $p \times 832$ somas.

```
versão paralela: trabalho  $O(n + p \cdot t \cdot w) = O(n)$  | tempo com p núcleos:  $O(n/p + p \cdot t \cdot w)$ 
```

04

Arquitetura da solução

Onde a operação vive no sistema em camadas e o caminho de uma requisição.

Apresentação

Página `index.html` servida pelo Spring em `/`. Gera a base, dispara as comparações, roda a bateria de medições, desenha a linha do tempo das threads e os gráficos (Chart.js). Mede o tempo de resposta do endpoint no navegador.

Aplicação (API)

Spring Boot. Controllers REST (`api`) recebem a requisição, o `DemandaService` escolhe a versão, e o `ProcessadorDemanda` (pacote `core`) executa o algoritmo sequencial ou paralelo. **É aqui que as threads trabalham.**

Dados

Histórico de requisições. No projeto, uma base sintética e reprodutível gerada em memória (`GeradorDados`), que representa o snapshot que em produção viria do banco. Isola na medição exatamente o que a atividade quer: o processamento.

4.1 Caminho de uma requisição

1. O navegador chama `GET /api/demanda/preditiva?modo=plataforma&threads=4`.
2. O `DemandaController` converte o parâmetro `modo` para o enum `Modo` e chama o `DemandaService`.
3. O serviço pega a base carregada (uma `AtomicReference`) e delega ao `ProcessadorDemanda`.
4. O processador **divide** os `n` registros em 4 fatias, **submete** 4 tarefas ao pool, cada uma com o seu `Acumulador`, **espera** todas (`invokeAll`), **soma** os parciais e chama a `AnaliseDemanda`.
5. O resultado (projeções + métricas + linha do tempo de cada thread) volta como JSON; o Jackson serializa os `records` automaticamente.

Decisão de projeto: o algoritmo não depende do Spring

Todo o algoritmo fica no pacote `core`, em Java puro. O Spring só expõe esse núcleo pela web. Isso permite testar o algoritmo isoladamente, reaproveitá-lo em outro contexto (um job em lote, por exemplo) e medir o processamento sem interferência do framework.

05

Estrutura do projeto

Organização de pastas e a responsabilidade de cada arquivo.

```
hemotrack-demanda/
├── pom.xml                Spring Boot 3.3.4, Java 21, -Xmx3g
├── README.md              como rodar, endpoints e dicas de medição
├── docs/                  justificativa e análise
├── scripts/medir.sh       medição pelo terminal (curl)
├── src/
│   ├── main/java/br/com/hemotrack/
│   │   ├── HemotrackApplication.java  ponto de entrada do Spring Boot
│   │   ├── core/
│   │   │   ├── TipoSanguineo.java      ALGORITMO (Java puro)
│   │   │   │   ├── 8 tipos ABO/Rh e prevalência
│   │   │   ├── RegistroConsumo.java    uma requisição histórica (record)
│   │   │   ├── FusosBrasil.java        os 4 fusos horários do país
│   │   │   ├── BaseHistorica.java      histórico + estoque atual (record)
│   │   │   ├── GeradorDados.java       gera a base sintética e reprodutível
│   │   │   ├── Acumulador.java         laço O(n), soma dos parciais, SHA-256
│   │   │   ├── ProcessadorDemanda.java sequencial, plataforma, virtual, inseguro
│   │   │   ├── AnaliseDemanda.java      $\mu$ ,  $\sigma$ , tendência, projeção, P(ruptura)
│   │   │   └── Execucacao, FatiaExecutada, ResultadoDemanda, ProjecaoTipo resp.
│   │   ├── service/DemandaService.java guarda a base e escolhe a versão
│   │   └── api/                          controllers e tratador de erros
│   ├── main/resources/
│   │   ├── application.properties
│   │   └── static/index.html             interface de demonstração
│   └── test/java/.../ProcessadorDemandaTest.java  sequencial = paralelo
```

06

0 código, classe por classe

Como o algoritmo foi construído, do modelo de dados até as versões paralelas. Todos os trechos abaixo são do código-fonte entregue.

6.1 Modelo de dados

TipoSanguineo é um *enum* com os 8 tipos ABO/Rh. Cada tipo guarda o rótulo exibido e a prevalência aproximada na população brasileira, usada só pelo gerador de dados sintéticos. O `ordinal()` de cada tipo (0 a 7) vira o índice da linha no acumulador.

core/TipoSanguineo.java

```
public enum TipoSanguineo {  
    A_POS("A+", 0.340),  
    A_NEG("A-", 0.080),  
    B_POS("B+", 0.080),  
    B_NEG("B-", 0.020),  
    AB_POS("AB+", 0.025),  
    AB_NEG("AB-", 0.005),  
    O_POS("O+", 0.360),  
    O_NEG("O-", 0.090);  
}
```

RegistroConsumo é um *record* (imutável) que representa uma requisição histórica. O instante é guardado em **UTC** (`instanteUtcMs`), como fica gravado no banco; o fuso do hospital é um índice para `FusosBrasil.ZONAS` (Noronha, Recife/Brasília, Manaus e Rio Branco). Imutabilidade importa aqui: várias threads leem os mesmos registros ao mesmo tempo, e objetos que não mudam podem ser compartilhados com segurança, sem trava.

core/RegistroConsumo.java

```
public record RegistroConsumo(long id, int hospitalId, byte fuso, TipoSanguineo tipo,  
                             long instanteUtcMs, int bolsas, boolean atendida) {  
}
```

BaseHistorica agrupa a lista de registros (embrulhada com `Collections.unmodifiableList`), a data de início, o número de semanas e o estoque atual por tipo.

6.2 GeradorDados: a base sintética

Para medir com 100 mil, 1 milhão, 5 milhões e 10 milhões de registros, o projeto gera a base com um `SplittableRandom` de **semente fixa** (42). A mesma semente produz exatamente os mesmos dados, então as medições e a assinatura do resultado são reprodutíveis em qualquer máquina. O gerador simula um país real:

- **Distribuição ABO/Rh brasileira** (O+ 36%, A+ 34%, O- 9%, ...), sorteada pela distribuição acumulada.
- **5.000 hospitais**, cada um fixo em um dos 4 fusos (80% no fuso de Brasília).
- **104 semanas** a partir de um domingo fixo (15/09/2024), porque a semana epidemiológica começa no domingo.
- **Demanda crescente de O-**: para esse tipo o dia é o maior de dois sorteios, o que concentra registros no fim do período. É o doador universal, usado em emergências: o caso crítico da história.

- **Sazonalidade:** no fim de ano e no carnaval, 35% das requisições pedem uma bolsa a mais.
- **5% de cancelamentos,** que não contam como consumo.

core/GeradorDados.java | laço de geração

```
List<RegistroConsumo> lista = new ArrayList<>(quantidade);
for (int i = 0; i < quantidade; i++) {
    TipoSanguineo tipo = TipoSanguineo.TODOS[sortear(rnd.nextDouble(), acumTipo)];

    // 0- tem demanda crescente: o máximo de dois sorteios concentra registros no fim do período
    int dia = rnd.nextInt(DIAS);
    if (tipo == TipoSanguineo.O_NEG) dia = Math.max(dia, rnd.nextInt(DIAS));

    int bolsas = 1 + rnd.nextInt(3);
    int semanaDoAno = (dia / 7) % 52;
    boolean altaTemporada = semanaDoAno ≥ 13 && semanaDoAno ≤ 17 // fim de ano
        || semanaDoAno ≥ 22 && semanaDoAno ≤ 25; // carnaval
    if (altaTemporada && rnd.nextDouble() < 0.35) bolsas++;

    int hospital = 1 + rnd.nextInt(HOSPITAIS);
    long instante = inicioUtcMs + dia * MS_DIA + rnd.nextLong(MS_DIA); // gravado em UTC
    boolean atendida = rnd.nextDouble() ≥ 0.05; // 5% canceladas

    lista.add(new RegistroConsumo(i + 1L, hospital, fusoDoHospital[hospital], tipo, instante, bolsas,
    atendida));
}
```

O estoque atual de cada tipo é gerado como um fator × a demanda esperada no trimestre: 1,45 para A+, 1,00 para AB- (no limite) e 0,95 para O- (abaixo da demanda, portanto em risco).

6.3 Acumulador: o laço quente $O(n)$

É o coração do desempenho. Um `Acumulador` guarda uma matriz `long[8][104]` (tipo × semana) e três contadores. O método `processar` percorre uma fatia `[inicio, fim)` da lista. Para cada registro: descarta cancelados, converte o instante UTC para a **data local do hospital** (`Instant` → `atZone` → `toLocalDate`, que consulta as regras do fuso), calcula a semana com `Math.floorDiv` e soma as bolsas na célula.

core/Acumulador.java | processar (0(k) para uma fatia de k registros)

```
public void processar(List<RegistroConsumo> registros, int inicio, int fim) {
    for (int i = inicio; i < fim; i++) {
        RegistroConsumo r = registros.get(i);
        if (!r.atendida()) {
            registrosCancelados++;
            continue;
        }
        LocalDate dataLocal = Instant.ofEpochMilli(r.instanteUtcMs())
            .atZone(FusosBrasil.ZONAS[r.fuso()])
            .toLocalDate();
        long semana = Math.floorDiv(dataLocal.toEpochDay() - inicioEpochDay, 7);
        if (semana < 0 || semana >= semanas) {
            foraDaJanela++;
            continue;
        }
        bolsas[r.tipo().ordinal()][(int) semana] += r.bolsas();
        registrosValidos++;
    }
}
```

Por que a conversão de fuso importa

Uma requisição feita às 23h em Manaus já é o dia seguinte em UTC. Sem converter, ela cairia na semana errada. Essa conversão é CPU pura, cerca de 49 ns por registro, e é o que torna o laço caro o bastante para valer paralelizar: com 10 milhões de registros são quase 0,5 s por consulta em uma thread.

O método `somar` é a **redução**: soma célula a célula outro acumulador neste. Custa 832 somas, independente de `n`. Como tudo é inteiro (`long`), a soma é associativa e comutativa, e a ordem em que os parciais chegam não muda o resultado. Se usássemos `double`, a ordem das somas poderia mudar o último dígito e a versão paralela divergiria da sequencial sem haver bug.

core/Acumulador.java | somar (redução)

```
public void somar(Acumulador outro) {
    for (int t = 0; t < bolsas.length; t++) {
        long[] destino = bolsas[t];
        long[] origem = outro.bolsas[t];
        for (int s = 0; s < semanas; s++) destino[s] += origem[s];
    }
    registrosValidos += outro.registrosValidos;
    registrosCancelados += outro.registrosCancelados;
    foraDaJanela += outro.foraDaJanela;
}
```

Por fim, `assinatura()` calcula um **SHA-256** de todos os números agregados e devolve os 16 primeiros caracteres. É a prova objetiva de que duas execuções deram a mesma resposta: basta comparar duas strings.

6.4 ProcessadorDemanda: versão sequencial

Uma única thread (a thread HTTP do Tomcat que atendeu a requisição) processa a base inteira com um só acumulador, depois chama a análise estatística. Os `System.nanoTime()` medem cada etapa separadamente, e esses tempos voltam na resposta.

core/ProcessadorDemanda.java | sequencial

```
public Execucão sequencial(BaseHistorica base) {
    long t0 = System.nanoTime();
    Acumulador total = new Acumulador(base.semanas(), base.inicio());
    total.processar(base.registros(), 0, base.tamanho());
    long t1 = System.nanoTime();

    List<FatiaExecutada> linha = List.of(new FatiaExecutada(0, Thread.currentThread().getName(),
        false, 0, base.tamanho(), 0, ms(t1 - t0)));

    long t2 = System.nanoTime(); // não há agregação na versão sequencial
    ResultadoDemanda resultado = AnaliseDemanda.analisar(total, base.estoqueAtual());
    long t3 = System.nanoTime();

    return new Execucão("sequencial", 1, base.tamanho(), ms(t3 - t0),
        ms(t1 - t0), ms(t2 - t1), ms(t3 - t2), linha, resultado);
}
```

6.5 ProcessadorDemanda: versão paralela com ExecutorService

O método `paralelo` implementa o padrão **dividir → mapear → reduzir**, pedido pela atividade ("cada thread processando a própria fatia, com agregação ao final"):

1. **Dividir:** a fatia i vai de $n \cdot i / p$ a $n \cdot (i+1) / p$. A conta é feita em `long` por precaução contra estouro de `int` em bases muito grandes, e a fórmula garante fatias contíguas, sem sobreposição e sem buracos (o que também é testado).
2. **Mapear:** cada tarefa (`Callable`) cria o **seu próprio** `Acumulador` e processa só a sua fatia. Nada é compartilhado para escrita.
3. **Esperar:** `invokeAll` submete as p tarefas ao pool e bloqueia até todas terminarem.
4. **Reduzir:** soma os parciais sempre na mesma ordem (fatia 0, 1, 2...).
5. **Analisar:** a etapa estatística roda uma vez, sobre o total.

core/ProcessadorDemanda.java | paralelo

```
private Execucão paralelo(String modo, BaseHistorica base, int p, ExecutorService executor) {
    validarThreads(p);
    long t0 = System.nanoTime();
    int n = base.tamanho();

    // 1. DIVIDIR: p tarefas, cada uma com a SUA fatia e o SEU acumulador
    List<Callable<Parcial>> tarefas = new ArrayList<>(p);
    for (int i = 0; i < p; i++) {
        final int indice = i;
        final int inicio = (int) ((long) n * i / p);
        final int fim = (int) ((long) n * (i + 1) / p);
        tarefas.add(() -> {
            long ini = System.nanoTime();
            Acumulador local = new Acumulador(base.semanas(), base.inicio());
            local.processar(base.registros(), inicio, fim); // 2. MAPEAR
            long fimNs = System.nanoTime();
            Thread atual = Thread.currentThread();
            return new Parcial(local, new FatiaExecutada(indice, nomeThread(atual), atual.isVirtual(),
                inicio, fim, ms(ini - t0), ms(fimNs - t0)));
        });
    }

    List<Parcial> parciais = executarTodas(executor, tarefas);
    long t1 = System.nanoTime();

    // 3. REDUZIR: sempre na mesma ordem (fatia 0, 1, 2...) - determinístico
    Acumulador total = new Acumulador(base.semanas(), base.inicio());
    List<FatiaExecutada> linha = new ArrayList<>(p);
    for (Parcial parcial : parciais) {
        total.somar(parcial.acumulador());
        linha.add(parcial.fatia());
    }
    long t2 = System.nanoTime();

    // 4. ANALISAR
    ResultadoDemanda resultado = AnaliseDemanda.analisar(total, base.estoqueAtual());
    long t3 = System.nanoTime();

    return new Execucão(modo, p, n, ms(t3 - t0), ms(t1 - t0), ms(t2 - t1), ms(t3 - t2), linha,
        resultado);
}
```

Cada tarefa também registra em que momento (ms desde o início da requisição) começou e terminou, qual thread a executou e qual fatia processou (`FatiaExecutada`). É isso que a interface desenha como linha do tempo.

Os pools de threads

Criar threads custa caro. Por isso há **um pool fixo por quantidade de threads** (2, 4, 8), criado na primeira chamada e reaproveitado (`computeIfAbsent` em um `ConcurrentHashMap`). O `prestartAllCoreThreads()` cria as threads antes da primeira medição, então o tempo medido é de processamento, não de criação de threads. As threads têm nomes legíveis (`hemo-p4-worker-1`) e são *daemon*, para não impedir o encerramento da JVM.

core/ProcessadorDemanda.java | pool reaproveitado

```
private ExecutorService pool(int threads) {
    validarThreads(threads);
    return pools.computeIfAbsent(threads, qtd → {
        AtomicInteger contador = new AtomicInteger(1);
        ThreadPoolExecutor executor = new ThreadPoolExecutor(qtd, qtd, 0L, TimeUnit.MILLISECONDS,
            new LinkedBlockingQueue<>(), r → {
                Thread t = new Thread(r, "hemo-p" + qtd + "-worker-" + contador.getAndIncrement());
                t.setDaemon(true);
                return t;
            });
        executor.prestartAllCoreThreads(); // threads já criadas: não medimos o custo de criação
        return executor;
    });
}
```

core/ProcessadorDemanda.java | executarTodas

```
private static List<Parcial> executarTodas(ExecutorService executor, List<Callable<Parcial>> tarefas) {
    try {
        List<Future<Parcial>> futuros = executor.invokeAll(tarefas); // bloqueia até todas terminarem
        List<Parcial> resultado = new ArrayList<>(futuros.size());
        for (Future<Parcial> f : futuros) resultado.add(f.get());
        return resultado;
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        throw new IllegalStateException("Processamento interrompido", e);
    } catch (ExecutionException e) {
        throw new IllegalStateException("Falha em uma das threads", e.getCause());
    }
}
```

O tratamento de exceções segue a boa prática: se a thread for interrompida, o sinal de interrupção é restaurado (`Thread.currentThread().interrupt()`); se uma tarefa falhar, a causa original é repassada.

6.6 Versão com threads virtuais (Java 21)

É o mesmo método `paralelo`, trocando só o executor: `Executors.newThreadPerTaskExecutor` com uma fábrica `Thread.ofVirtual()`. Cada fatia ganha uma thread virtual nova. O `try-with-resources` fecha o executor, o que espera as tarefas terminarem.

core/ProcessadorDemanda.java | virtual

```
public Execucão virtual(BaseHistorica base, int threads) {
    ThreadFactory fabrica = Thread.ofVirtual().name("hemo-virtual-", 1).factory();
    try (ExecutorService executor = Executors.newThreadPerTaskExecutor(fabrica)) {
        return paralelo("virtual", base, threads, executor);
    }
}
```

6.7 A versão errada de propósito (race condition)

Para mostrar por que os acumuladores locais são necessários, o modo `inseguro` faz todas as threads somarem no **mesmo** acumulador, sem sincronização:

`core/ProcessadorDemanda.java` | `inseguro` (NÃO usar em produção)

```
tarefas.add(() -> {  
    long ini = System.nanoTime();  
    compartilhado.processar(base.registros(), inicio, fim); // ← estado compartilhado!  
    long fimNs = System.nanoTime();  
    return new Parcial(null, new FatiaExecutada(indice, nomeThread(Thread.currentThread()), false,  
        inicio, fim, ms(ini - t0), ms(fimNs - t0)));  
});
```

A operação `bolsas[t][s] += b` parece uma coisa só, mas são três passos: ler o valor, somar e gravar. Se duas threads leem o mesmo valor antes de qualquer uma gravar, uma gravação apaga a outra e aquela soma se perde. O resultado muda a cada execução (seção 14).

6.8 AnaliseDemanda: a estatística da US07

Com as 8 séries semanais prontas, a análise calcula, para cada tipo:

$\mu = (\sum x_t) / w$	média semanal
$\sigma = \sqrt{(\sum (x_t - \mu)^2 / (w - 1))}$	desvio-padrão amostral
$b = \sum (t - t')(x_t - \mu) / \sum (t - t')^2$	tendência (regressão linear)
$D = \sum_{t=w..w+12} \max(0, \mu + b \cdot (t - t'))$	demanda projetada no trimestre
$\sigma_T = \sigma \cdot \sqrt{13}$	incerteza do trimestre
$P(\text{ruptura}) = 1 - \Phi((\text{estoque} - D) / \sigma_T)$	

Φ é a função de distribuição acumulada da Normal padrão, calculada pela aproximação de Abramowitz & Stegun (erro menor que $1,5 \times 10^{-7}$). O risco é **alto** se $P \geq 50\%$, **moderado** se $P \geq 20\%$ e **baixo** nos demais casos; tipos não baixos entram em `tiposEmAlerta`.

core/AnaliseDemanda.java | laço por tipo

```
for (TipoSanguineo tipo : TipoSanguineo.TODOS) {
    long[] x = total.serie(tipo);

    long soma = 0;
    for (long v : x) soma += v;
    double media = (double) soma / w;

    double somaQuad = 0, cov = 0;
    for (int t = 0; t < w; t++) {
        double d = x[t] - media;
        somaQuad += d * d;
        cov += (t - tMedio) * d;
    }
    double desvio = Math.sqrt(somaQuad / (w - 1));
    double inclinacao = cov / somaQuadT;

    double projecao = 0;
    for (int t = w; t < w + SEMANAS_PROJECAO; t++) {
        projecao += Math.max(0, media + inclinacao * (t - tMedio));
    }
    double desvioTrimestre = desvio * Math.sqrt(SEMANAS_PROJECAO);

    long estoque = estoqueAtual[tipo.ordinal()];
    double z = desvioTrimestre == 0 ? Double.POSITIVE_INFINITY : (estoque - projecao) / desvioTrimestre;
    double pRuptura = 1.0 - phi(z);
    String risco = pRuptura >= 0.5 ? "ALTO" : pRuptura >= 0.2 ? "MODERADO" : "BAIXO";
    if (!"BAIXO".equals(risco)) alertas.add(tipo.rotulo());
}
```

Por que essa etapa não é paralelizada

Ela custa $O(t \cdot w) = O(8 \times 104)$, independente de n : menos de 1 ms. Paralelizá-la não traria ganho. É a "parte serial" da Lei de Amdahl, junto com a recepção HTTP, a redução e a serialização do JSON.

6.9 Camada Spring: serviço e controllers

O `DemandaService` (`@Service`) guarda a base em uma `AtomicReference`, para que a troca de base seja atômica e visível a todas as threads HTTP, e escolhe a versão com um `switch` sobre o enum `Modo`. O `@PreDestroy` encerra os pools quando a aplicação para. Ao gerar uma nova base ele chama `System.gc()` antes, para não medir a coleta de lixo da base anterior depois.

service/DemandaService.java

```
public Execucão executar(Modo modo, int threads) {
    BaseHistorica b = base.get();
    if (b == null) throw new IllegalStateException("Nenhuma base carregada. Gere os dados em POST /api/dados.");
    return switch (modo) {
        case SEQUENCIAL → processador.sequencial(b);
        case PLATAFORMA → processador.plataforma(b, threads);
        case VIRTUAL → processador.virtual(b, threads);
        case INSEGURO → processador.inseguro(b, threads);
    };
}
```

O `DemandaController` expõe a análise e a verificação de igualdade. O `TratadorErros` (`@RestControllerAdvice`) transforma exceções em respostas HTTP claras: 409 quando não há base carregada, 400 para modo ou número de threads inválido e 507 se faltar memória.

api/DemandaController.java | preditiva

```
@GetMapping("/preditiva")
public Execucão preditiva(@RequestParam(defaultValue = "sequencial") String modo,
    @RequestParam(defaultValue = "1") int threads) {
    return service.executar(Modo.valueOf(modo.toUpperCase()), threads);
}
```

api/DemandaController.java | verificar

```
@GetMapping("/verificar")
public Map<String, Object> verificar(@RequestParam(defaultValue = "8") int threads) {
    String referencia = service.executar(Modo.SEQUENCIAL, 1).resultado().assinatura();
    Map<String, Object> r = new LinkedHashMap<>();
    r.put("sequencial", referencia);
    boolean todasIguais = true;
    for (int t : new int[]{2, 4, threads}) {
        String p = service.executar(Modo.PLATAFORMA, t).resultado().assinatura();
        String v = service.executar(Modo.VIRTUAL, t).resultado().assinatura();
        r.put("plataforma_" + t, p);
        r.put("virtual_" + t, v);
        todasIguais &= referencia.equals(p) && referencia.equals(v);
    }
    r.put("todasIguais", todasIguais);
    return r;
}
```

6.10 Testes automatizados

`ProcessadorDemandaTest` gera uma base de 300 mil registros e verifica que o **resultado inteiro** (o *record* `ResultadoDemanda`, comparado campo a campo pelo `equals`) é igual ao sequencial para 2, 3, 4, 7, 8 e 16 threads de plataforma e 2, 4 e 8 virtuais. Quantidades ímpares (3, 7) testam fatias de tamanhos diferentes. Outros testes garantem que as fatias cobrem a base sem sobreposição, que válidos + cancelados + fora da janela = n, e que 0- aparece em alerta.

```
test/.../ProcessadorDemandaTest.java
```

```
@ParameterizedTest
@ValueSource(ints = {2, 3, 4, 7, 8, 16})
void threadsDePlataformaDevolveOMesmoResultado(int threads) {
    assertEquals(referencia, processador.plataforma(base, threads).resultado());
}
```

07

Endpoints da API

Contrato HTTP do serviço.

Endpoint	O que faz
POST /api/dados?tamanho=&semente=	Gera a base histórica (1 mil a 20 milhões de registros; padrão 1 milhão, semente 42) e devolve tamanho, período e tempo de geração.
GET /api/dados	Informa se há base carregada e o seu tamanho.
GET /api/demanda/preditiva?modo=&threads=	A US07. modo : sequencial, plataforma, virtual ou inseguro; threads : 1 a 64. Devolve a projeção e as métricas de execução.
GET /api/demanda/verificar?threads=8	Roda sequencial, plataforma e virtual (2, 4 e N threads) e compara as assinaturas: todasIguais: true prova ausência de race condition.
GET /api/sistema	Núcleos visíveis pela JVM, versão do Java, sistema operacional e heap máximo, para registrar a máquina de teste no relatório.

7.1 Exemplo de resposta

Resposta resumida de GET /api/demanda/preditiva?modo=plataforma&threads=4 com 1 milhão de registros. Os campos de tempo permitem separar o processamento das fatias, a redução e a análise:

resposta JSON (resumida)

```
{
  "modo": "plataforma", "threads": 4, "tamanho": 1000000,
  "tempoProcessamentoMs": 29.11, "tempoFatiasMs": 28.9,
  "tempoAgregacaoMs": 0.013, "tempoAnaliseMs": 0.19,
  "fatias": [
    {"indice": 0, "thread": "hemo-p4-worker-3", "virtual": false,
      "inicioIndice": 0, "fimIndice": 250000, "inicioMs": 0.04, "fimMs": 27.8},
    "... (uma entrada por thread)"
  ],
  "resultado": {
    "registrosValidos": 950114, "registrosCancelados": 49745,
    "foraDaJanela": 141, "totalBolsas": 1955170, "semanasAnalisadas": 104,
    "projecoes": [
      {"tipo": "0-", "mediaSemanal": 1687.3462, "desvioSemanal": 963.4833,
        "tendenciaSemanal": 31.6448, "demandaTrimestre": 46001.3696,
        "desvioTrimestre": 3473.8886, "estoqueAtual": 20921,
        "probabilidadeRuptura": 1.0, "risco": "ALTO"},
      "... (8 tipos)"
    ],
    "tiposEmAlerta": ["AB-", "0-"],
    "assinatura": "1ff7dd2efde0e8aa"
  }
}
```

7.2 Exemplos de uso pelo terminal

terminal

```
curl -X POST "localhost:8080/api/dados?tamanho=1000000&semente=42"
curl "localhost:8080/api/demanda/preditiva?modo=sequencial"
curl "localhost:8080/api/demanda/preditiva?modo=plataforma&threads=4"
curl "localhost:8080/api/demanda/preditiva?modo=virtual&threads=8"
curl "localhost:8080/api/demanda/preditiva?modo=inseguro&threads=8"
curl "localhost:8080/api/demanda/verificar?threads=8"
```

08

A interface de demonstração

Como a página mostra, de forma dinâmica, a diferença entre sem thread e com thread.

A página em `http://localhost:8080` segue o fluxo da atividade em quatro blocos: gerar a base, comparar sequencial e paralelo, medir e ver o que acontece sem acumuladores locais. No topo, a barra mostra os núcleos visíveis pela JVM, a versão do Java e se o servidor está no ar.



Figura 1. Topo da interface: dados da máquina, geração da base histórica e controles da comparação.

8.1 A linha do tempo das threads

Ao clicar em **Comparar**, a página aquece o JIT (3 execuções de cada versão fora da medição), roda a versão sequencial e depois a paralela. Cada barra mostra **quando uma thread começou e terminou a sua fatia**, na mesma escala de tempo. A animação reproduz os tempos reais em câmera lenta: a barra sequencial é uma só e longa; as paralelas são várias e mais curtas. Abaixo, quatro indicadores: speedup, eficiência por thread, tempo da redução e a assinatura SHA-256 das duas respostas (**Idênticas**).

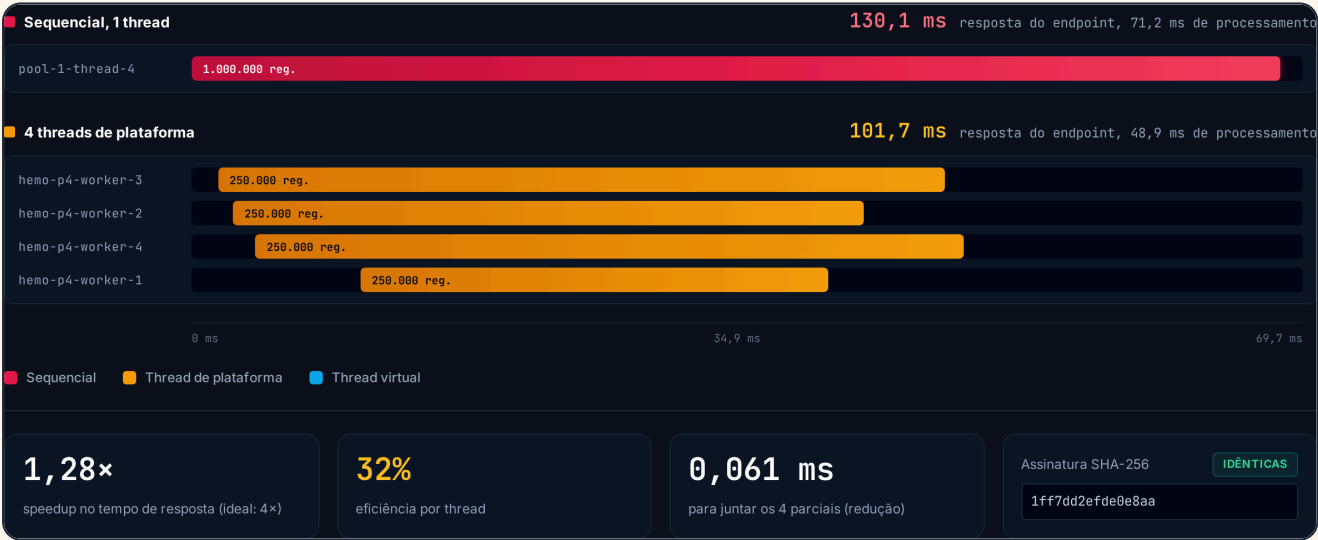


Figura 2. Linha do tempo sequencial x 4 threads. Esta captura foi feita no ambiente de desenvolvimento, que tinha **1 núcleo**: as 4 threads se revezam no mesmo núcleo (barras escalonadas e sobrepostas) e o ganho é pequeno. Na máquina do grupo, com 16 núcleos, as barras começam juntas e terminam em cerca de um terço do tempo sequencial. A diferença entre as duas situações é justamente concorrência x paralelismo (seção 12).

8.2 Bateria de medições, projeção e race condition

- **Bateria de medições:** escolhem-se os tamanhos, o número de repetições e se inclui threads virtuais. Para cada tamanho, a página gera a base, aquece o JIT (5 execuções de cada versão) e mede cada configuração N vezes com `performance.now()` ao redor do `fetch`, ou seja, o **tempo de resposta do endpoint** visto pelo

cliente. Mostra mediana, mínimo, tempo de processamento no servidor, speedup, eficiência e se o resultado foi igual ao sequencial. Exporta CSV e tabela em Markdown.

- **Projeção para o próximo trimestre:** o gráfico $\mu \pm \sigma$ e a tabela da US07 (Figura 3).
- **Race condition:** roda 5 vezes a versão insegura e uma vez a correta, mostrando bolsas perdidas e assinaturas divergentes.

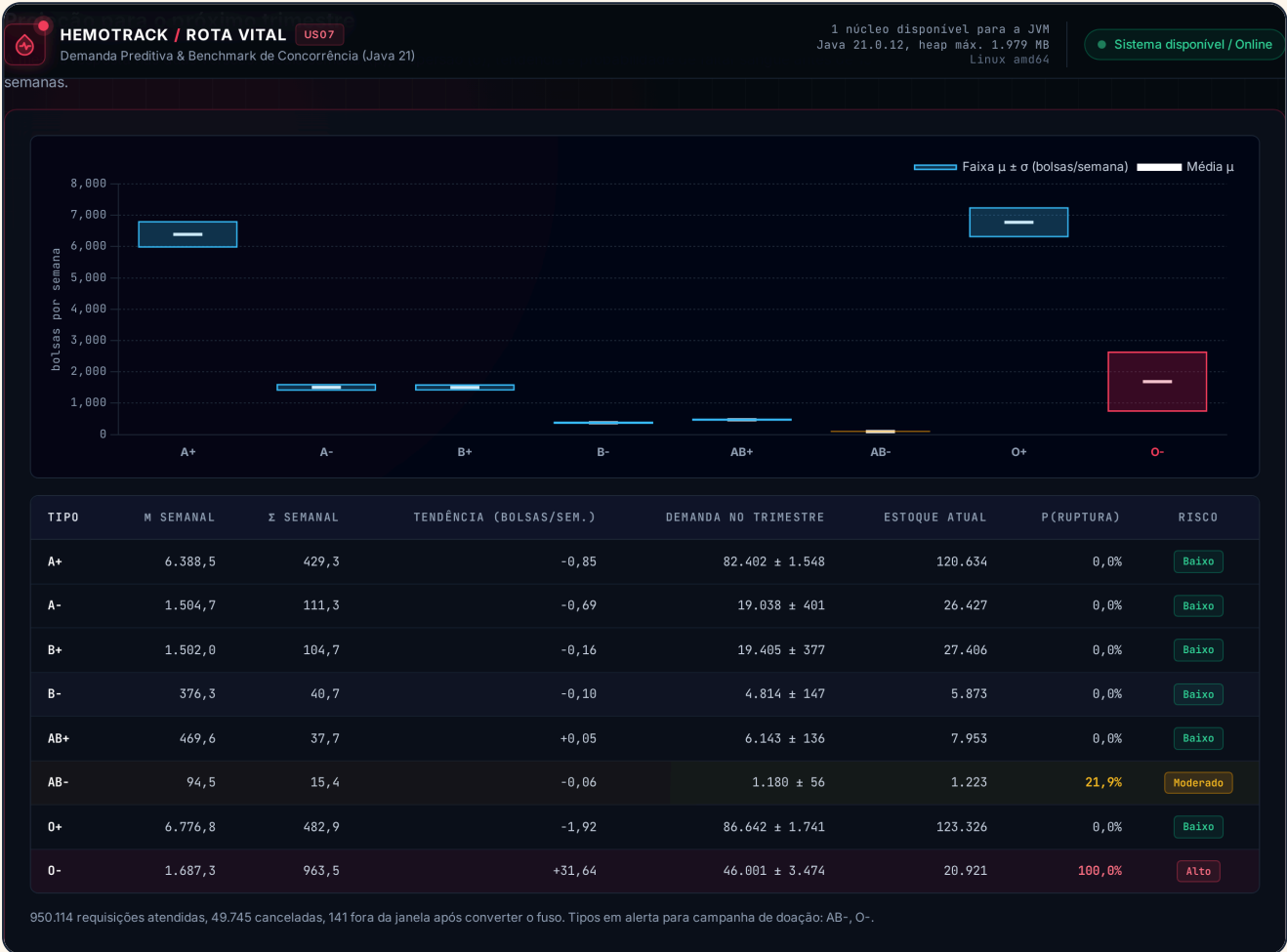


Figura 3. Módulo "Demanda Preditiva" na interface: faixas $\mu \pm \sigma$ por tipo e tabela com tendência, demanda do trimestre, estoque, P(ruptura) e risco. O- em risco alto e AB- em risco moderado, como pede o critério de aceite.

09

Como executar e medir

Passo a passo para reproduzir tudo em outra máquina.

9.1 Pré-requisitos

JDK 21(as threads virtuais exigem Java 21) e **Maven 3.9+**, ou uma IDE como IntelliJ ou VS Code com suporte a Maven.

9.2 Subir a aplicação

terminal

```
cd hemotrack-demanda
mvn spring-boot:run          # já sobe com -Xmx3g (configurado no pom.xml)
# abrir http://localhost:8080
```

Pela IDE, basta rodar `HemotrackApplication`. Para 5 ou 10 milhões de registros, adicione `-Xmx3g` em VM options: 10 milhões de objetos ocupam cerca de 500 MB.

9.3 Rodar os testes

terminal

```
mvn test
```

9.4 Medir

Na página, bloco 3 (Bateria de medições): marcar 100 mil e 1 milhão (e, se a máquina tiver memória, 5 e 10 milhões), 10 repetições, incluir threads virtuais e clicar em **Rodar medições**. Ao final, **Baixar CSV** ou **Copiar tabela em Markdown**. Pelo terminal (Linux, macOS ou Git Bash): `./scripts/medir.sh 10`, que gera `medicoes.csv` medindo o endpoint com `curl`.

Cuidados para medições confiáveis

- Notebook na tomada e outros programas fechados.
- Usar a **mediana**, não a média: pausas do coletor de lixo ou do sistema operacional distorcem a média.
- Não medir a primeira execução: o JIT ainda está compilando o código (a página aquece antes).
- Repetir a bateria para ver se os números se repetem.

10

Resultados de negócio: a projeção da US07

O que o analista vê para o próximo trimestre (base de 1 milhão de requisições, semente 42).

950.114

requisições atendidas analisadas

1.955.170

bolsas consumidas em 104 semanas

0-

risco alto: P(ruptura)= 100%

AB-

risco moderado: P(ruptura)= 21,9%

Tipo	μ semanal	σ semanal	Tendência	Demanda no trimestre	Estoque	P(ruptura)	Risco
A+	6.388,5	429,3	-0,85	82.402 $\pm$ 1.548	120.634	0,0%	Baixo
A-	1.504,7	111,3	-0,69	19.038 $\pm$ 401	26.427	0,0%	Baixo
B+	1.502,0	104,7	-0,16	19.405 $\pm$ 377	27.406	0,0%	Baixo
B-	376,3	40,7	-0,10	4.814 $\pm$ 147	5.873	0,0%	Baixo
AB+	469,6	37,7	+0,05	6.143 $\pm$ 136	7.953	0,0%	Baixo
AB-	94,5	15,4	-0,06	1.180 $\pm$ 56	1.223	21,9%	Moderado
O+	6.776,8	482,9	-1,92	86.642 $\pm$ 1.741	123.326	0,0%	Baixo
O-	1.687,3	963,5	+31,64	46.001 $\pm$ 3.474	20.921	100,0%	Alto

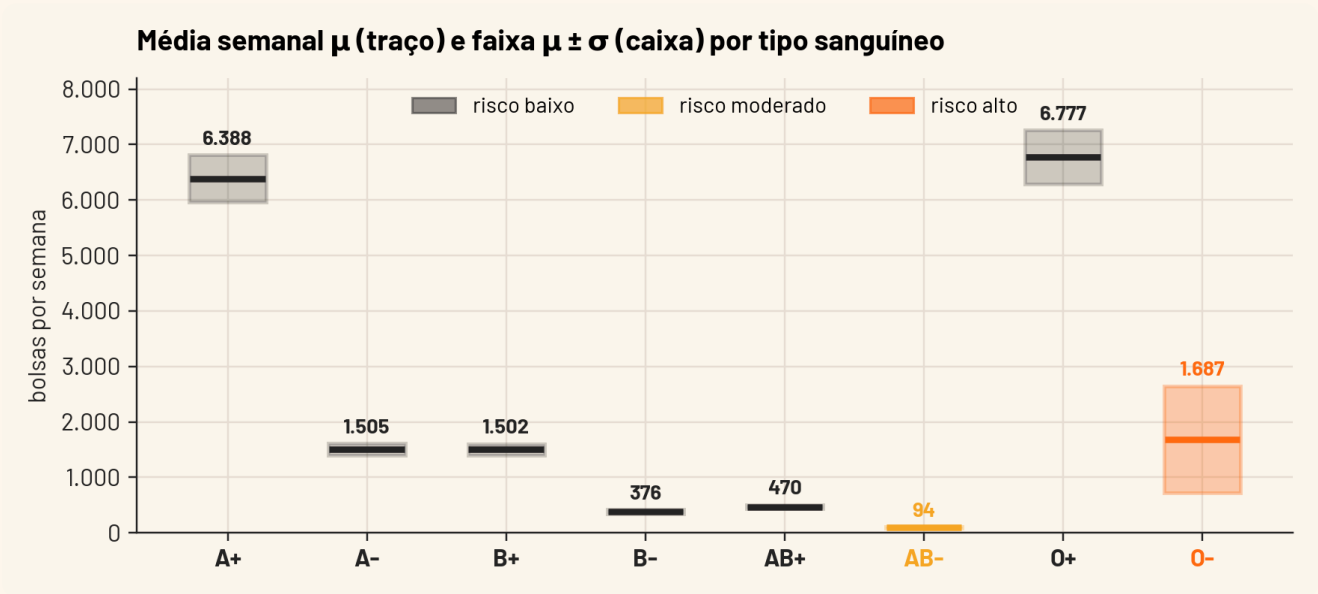


Figura 4. O "Então" do critério de aceite: média estimada μ (traço) e faixa de dispersão $\mu \pm \sigma$ (caixa) para cada tipo. O- tem a maior dispersão relativa ($\sigma \approx 57\%$ de μ) porque o consumo cresce ao longo do período.

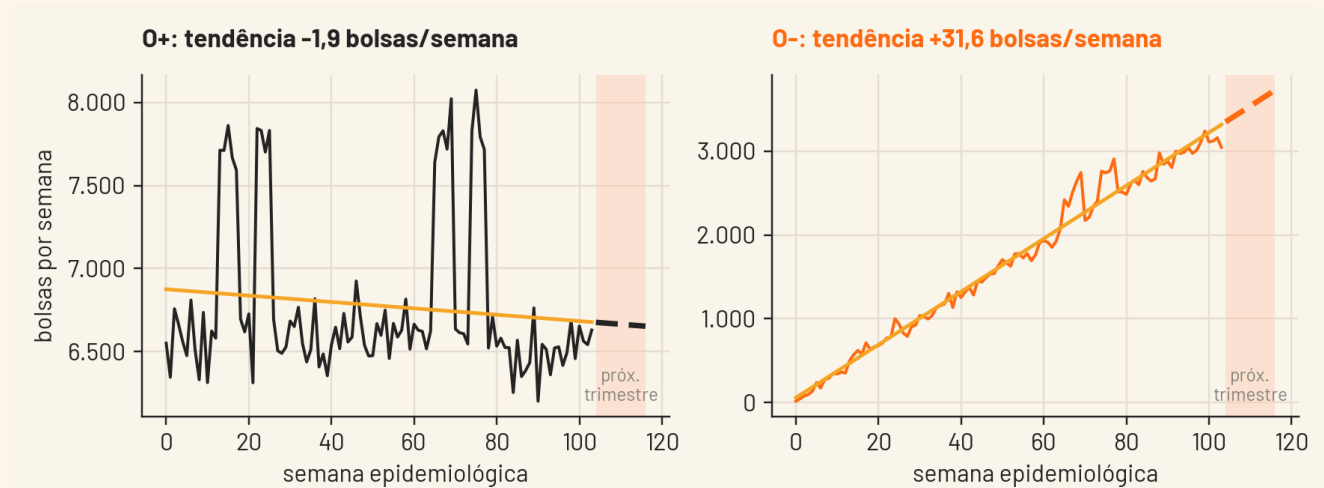


Figura 5. Série semanal de O+ e O-, com a reta de regressão (âmbar) e a projeção do próximo trimestre (tracejado). O+ é estável, com picos sazonais de fim de ano e carnaval; O- cresce cerca de 31,6 bolsas por semana, e a projeção leva em conta essa tendência.

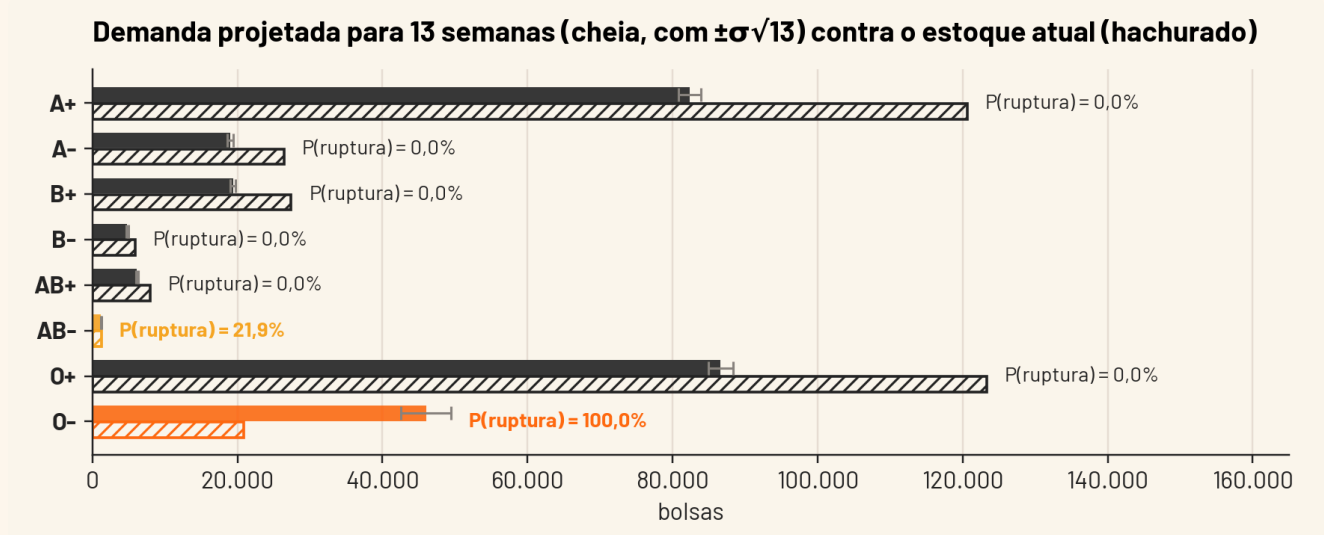


Figura 6. O "E" destaca os tipos com risco de ruptura": demanda projetada para 13 semanas (barra cheia, com $\pm \sigma\sqrt{13}$) contra o estoque atual (hachurado). Para O-, a demanda projetada (≈ 46 mil bolsas) é mais que o dobro do estoque (≈ 21 mil): ruptura praticamente certa. AB- está no limite, com 21,9% de chance.

Leitura para o analista

A campanha de doação do próximo trimestre deve priorizar **O-** (ruptura quase certa, demanda crescente) e acompanhar **AB-** (estoque no limite). Os demais tipos têm estoque com folga de 25% a 45% sobre a demanda projetada.

11

Medições de desempenho

Tempo de resposta do endpoint por versão, número de threads e tamanho da entrada.

11.1 Máquina e metodologia

Máquina do grupo	16 núcleos visíveis para a JVM, Windows 11(amd64)
JVM	Java 21.0.12.1, heap máximo de 8.116 MB
O que foi medido	Tempo de resposta do endpoint no navegador(<code>performance.now()</code> ao redor do <code>fetch</code>) e tempo de processamento informado pelo servidor
Repetições	10 por configuração, após 5 execuções de aquecimento do JIT; valor reportado = mediana
Speedup / eficiência	speedup = mediana sequencial ÷ mediana da versão; eficiência = speedup ÷ threads

11.2 Tabela de resultados

Registros	Versão	Threads	Resposta (mediana)	Resposta (mínimo)	Processamento	Speedup	Eficiência	vs. seq.
100.000	sequencial	1	14,60 ms	12,80 ms	9,93 ms	1,00×	100%	igual
100.000	plataforma	2	12,95 ms	9,40 ms	6,24 ms	1,13×	56%	igual
100.000	plataforma	4	12,70 ms	9,60 ms	5,07 ms	1,15×	29%	igual
100.000	plataforma	8	10,80 ms	8,90 ms	4,56 ms	1,35×	17%	igual
100.000	virtual	2	13,20 ms	11,00 ms	6,56 ms	1,11×	55%	igual
100.000	virtual	4	12,70 ms	10,60 ms	5,54 ms	1,15×	29%	igual
100.000	virtual	8	21,35 ms	13,00 ms	6,66 ms	0,68×	9%	igual
1.000.000	sequencial	1	90,05 ms	51,90 ms	85,23 ms	1,00×	100%	igual
1.000.000	plataforma	2	57,25 ms	42,80 ms	52,97 ms	1,57×	79%	igual
1.000.000	plataforma	4	33,60 ms	17,80 ms	29,11 ms	2,68×	67%	igual

A bateria executou 14 configurações; a tabela registrada pelo grupo traz as 10 acima. As configurações restantes de 1 milhão(8 threads de plataforma e as virtuais) podem ser completadas com o CSV exportado pela interface. Em todas as linhas o resultado foi igual ao sequencial (mesma assinatura SHA-256).

2,68×

speedup do endpoint com 4 threads, 1 milhão de registros

2,93×

speedup só do processamento (85,2 → 29,1 ms)

62,7%

menos tempo de resposta (90,1 → 33,6 ms)

1,35×

melhor speedup com 100 mil registros(8 threads)

11.3 Gráficos

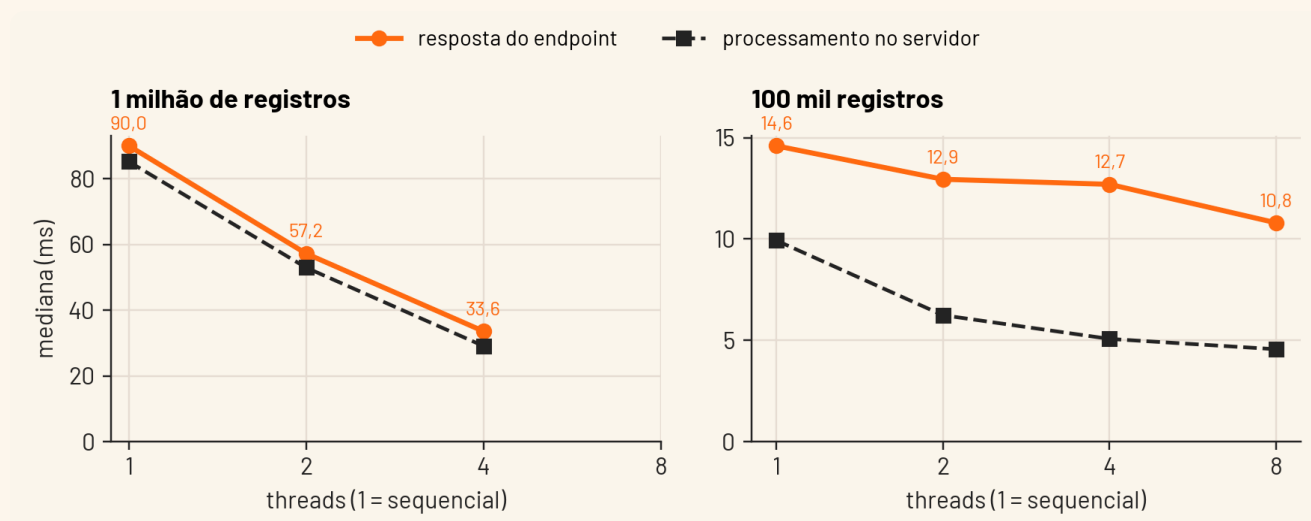


Figura 7. Tempo de resposta do endpoint (laranja) e tempo de processamento no servidor (preto) por número de threads. Com 1 milhão de registros as duas curvas descem juntas; com 100 mil, o processamento cai pela metade, mas a resposta quase não muda: o custo fixo da requisição HTTP domina.

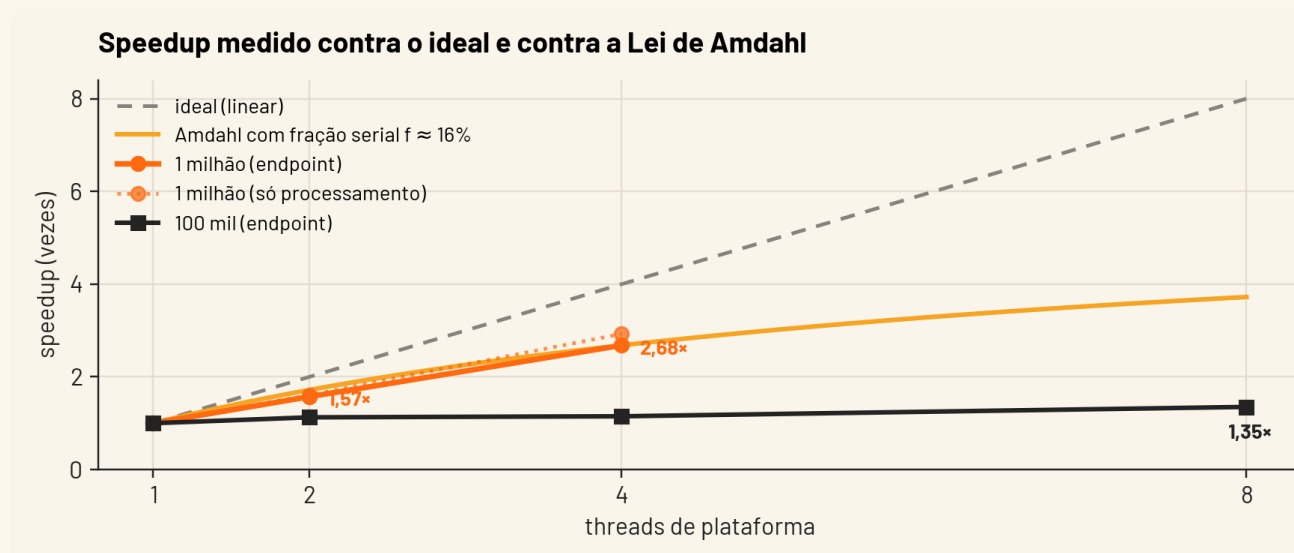


Figura 8. Speedup medido contra o ideal linear e contra a curva da Lei de Amdahl ajustada aos dados (fração serial $\approx 16\%$). Com 1 milhão de registros o ganho é real, mas abaixo do ideal; com 100 mil ele praticamente não aparece.

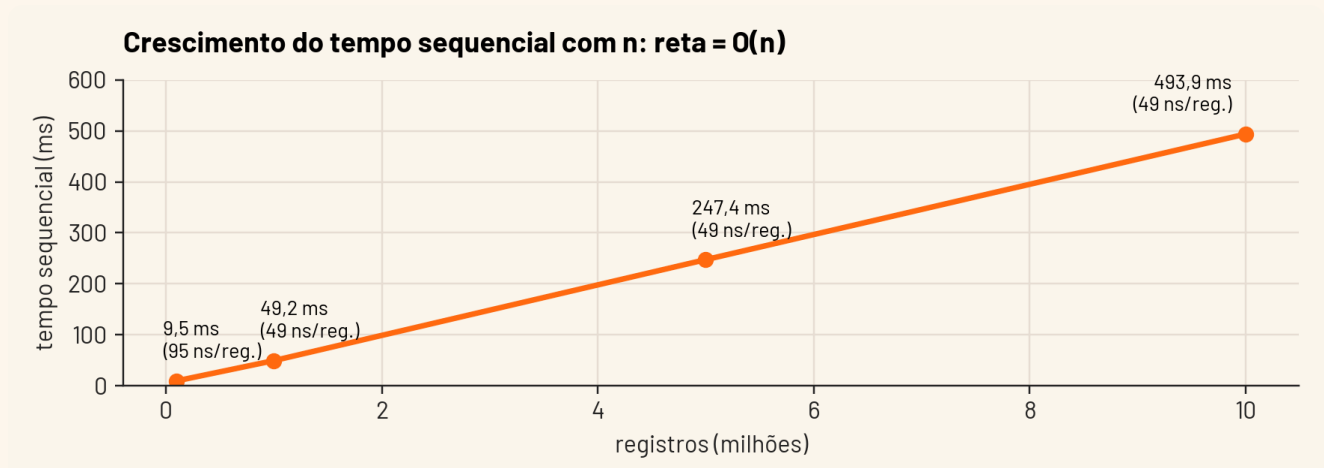


Figura 9. Tempo sequencial em função de n (medido no ambiente de desenvolvimento). A relação é uma reta com custo constante de ≈ 49 ns por registro a partir de 1 milhão: evidência empírica de que o algoritmo é $O(n)$. Em 100 mil o custo por registro é maior porque os custos fixos ainda pesam.

12

Análise dos resultados

As quatro perguntas da atividade, respondidas com os números medidos.

12.1 O ganho foi linear? Por que não?

Não. Com 1 milhão de registros o speedup foi **1,57× com 2 threads** e **2,68× com 4**, contra os ideais de $2\times$ e $4\times$ (eficiência de 79% e 67%). Com 100 mil registros ficou entre $1,13\times$ e $1,35\times$. Há quatro motivos, todos visíveis nos dados:

- **Lei de Amdahl: existe uma parte que não se divide.** Receber a requisição HTTP, somar os parciais, calcular μ , σ e a projeção, serializar o JSON e transmitir a resposta rodam em uma thread só. Ajustando a lei aos dados de 1 milhão, a fração serial fica em torno de **16%**. Com $f = 0,16$, o speedup máximo teórico, mesmo com infinitas threads, é $1/f \approx 6\times$, e com 8 threads seria $\approx 3,7\times$.
- **Custo fixo da requisição.** Com 100 mil registros o processamento caiu de 9,93 para 4,56 ms ($2,2\times$), mas a resposta só caiu de 14,6 para 10,8 ms, porque cerca de 5 ms são de HTTP, JSON e navegador, que não diminuem com threads. Paralelismo só compensa quando o trabalho é grande: em 1 milhão esse custo fixo é só ~5% do total.
- **Memória compartilhada.** As threads leem milhões de objetos da RAM. Todos os núcleos disputam o mesmo barramento e os mesmos níveis de cache; a partir de certo ponto a memória, e não a CPU, vira o limite.
- **Custo de coordenação e a thread mais lenta.** Despachar tarefas ao pool, esperar todas (`invokeAll`) e juntar os parciais tem custo, e o tempo total é o da fatia que termina por último. A linha do tempo da interface mostra fatias terminando em momentos diferentes. Núcleos lógicos (hyper-threading) também não dobram a capacidade real.

12.2 A Big-O mudou?

Não. O **trabalho total continua $O(n)$** : cada registro é visitado uma única vez, com ou sem threads (mais a redução, $O(p \cdot t \cdot w)$, desprezível). O que muda é o **tempo decorrido**, que passa a $O(n/p + p \cdot t \cdot w)$, porque o trabalho é repartido entre p núcleos. Threads mudam a **constante**, não a classe de complexidade: dobrar a base continua dobrando o tempo com qualquer número de threads (Figura 9). Paralelismo não transforma um algoritmo $O(n)$ em $O(\log n)$; ele divide o mesmo $O(n)$ entre mais trabalhadores.

12.3 Concorrência × paralelismo: Mesa DJ × HemoTrack

Mesa DJ: concorrência

Várias tarefas **independentes** chegando ao mesmo tempo (vários pedidos de música). O objetivo é que ninguém fique bloqueado esperando. Funciona até com um único núcleo: o sistema alterna entre as tarefas. É uma questão de **estrutura**: lidar com muitas coisas ao mesmo tempo.

HemoTrack US07: paralelismo

Uma única tarefa **grande**, dividida em pedaços que rodam **no mesmo instante** em núcleos diferentes. O objetivo é terminar mais rápido.

Exige vários núcleos. É uma questão de **execução**: fazer muitas coisas ao mesmo tempo.

Tivemos uma prova prática dessa diferença. No ambiente de desenvolvimento, com **1 núcleo**, 1 milhão de registros levou 49,2 ms sequencial e **50,3 ms com 4 threads**: as threads existiam e se revezavam (havia concorrência), mas não rodavam ao mesmo tempo (não havia paralelismo), então não houve ganho. Na máquina de 16 núcleos, o mesmo código com as mesmas 4 threads caiu de 90,1 para 33,6 ms. O código era idêntico; o que mudou foi o hardware disponível para executar as threads simultaneamente.

12.4 Quando nem 8 threads bastarem

O limite passa a ser uma máquina só: seus núcleos, sua memória e o próprio teto de Amdahl (~6× com a fração serial medida). O caminho é levar o mesmo padrão **dividir → mapear → reduzir** para **várias máquinas**, o tema da Unidade 2. A seção 15 detalha essa evolução.

13

Opcional: threads virtuais do Java 21

Comparação com as threads de plataforma no mesmo algoritmo.

As threads virtuais (Project Loom) são threads leves gerenciadas pela JVM, não pelo sistema operacional. Milhares delas são executadas sobre poucas **threads carregadoras** (um `ForkJoinPool` com uma thread por núcleo). A grande vantagem: quando uma thread virtual **bloqueia em I/O** (banco, rede), ela libera a carregadora, que passa a executar outra. Por isso servem para concorrência massiva com espera.

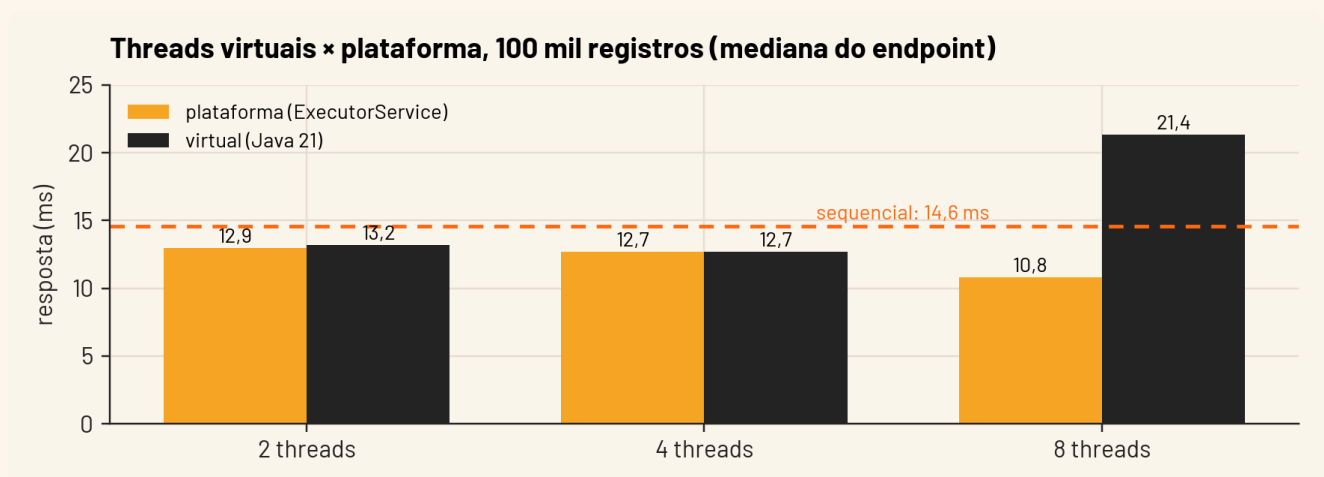


Figura 10. Threads virtuais × plataforma com 100 mil registros. Com 2 e 4 threads os tempos são praticamente iguais; com 8, a versão virtual foi pior que a sequencial (0,68×).

- **Com 2 e 4 threads o desempenho foi igual** ao das threads de plataforma (13,2 × 12,95 ms; 12,7 × 12,7 ms). Faz sentido: o trabalho é **CPU pura**, sem nenhuma espera de I/O. As threads virtuais rodam sobre o mesmo número de núcleos e não têm nenhum bloqueio para aproveitar.
- **Com 8 threads a versão virtual foi mais lenta** (21,35 ms). Em 100 mil registros cada fatia dura menos de 1 ms, então qualquer custo fixo pesa. No nosso código as threads de plataforma vêm de um pool já criado e reaproveitado, enquanto as virtuais são criadas a cada requisição (um executor novo por chamada), além do escalonamento e do *work stealing* do `ForkJoinPool`. O mínimo de 13,0 ms contra a mediana de 21,35 ms também indica alta variação nessa configuração.

Conclusão sobre threads virtuais

Threads virtuais **não aceleram cálculo**; elas **aumentam a quantidade de tarefas que podem esperar ao mesmo tempo**. Seriam a escolha certa para a US05 (milhares de sensores enviando telemetria) ou para atender milhares de requisições HTTP que esperam o banco. Para a US07, que é CPU pura, o pool fixo de threads de plataforma, do tamanho dos núcleos, é a ferramenta adequada.

14

Race condition: por que acumuladores locais

A versão insegura rodada 5 vezes na máquina do grupo (1 milhão de registros, 8 threads).

Execução	Bolsas contadas	Perdas	Requisições válidas	Assinatura	Status
Sequencial (referência)	1.955.170	0	950.114	1ff7dd2efde0e8aa	referência
Sem proteção, rodada 1	1.952.754	2.416	595.170	af3612e190e6a65b	diverge
Sem proteção, rodada 2	1.952.804	2.366	595.983	58508197121562c0	diverge
Sem proteção, rodada 3	1.952.732	2.438	599.927	87114f5a1167b3b9	diverge
Sem proteção, rodada 4	1.952.508	2.662	578.836	92b4b02141e369ec	diverge
Sem proteção, rodada 5	1.952.303	2.867	564.016	4d74f4d827509a52	diverge
8 threads com acumulador local	1.955.170	0	950.114	1ff7dd2efde0e8aa	igual

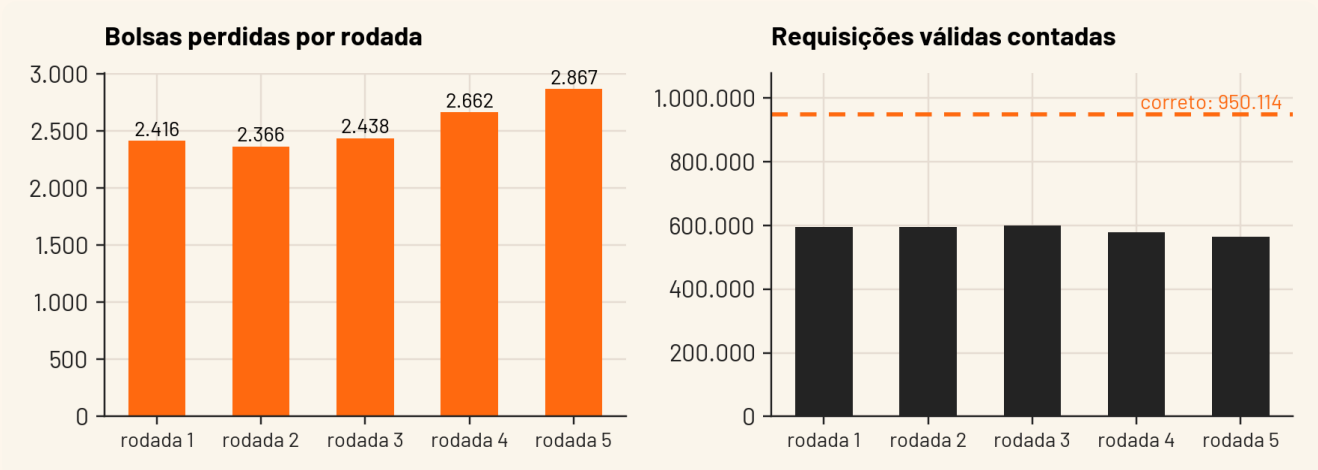


Figura 11. A versão sem proteção errou nas 5 rodadas, e cada rodada errou diferente (assinaturas distintas). A versão correta, com as mesmas 8 threads, acertou.

Por que o contador de requisições perdeu muito mais que as bolsas? As bolsas se espalham por 832 células diferentes: duas threads só colidem quando somam na mesma célula no mesmo instante, então perderam ~0,13% das bolsas. Já o contador `registrosValidos` é **um único campo** incrementado pelas 8 threads a cada registro: a disputa é máxima, e ~38% dos incrementos se perderam. Quanto mais concentrado o estado compartilhado, pior a race condition.

A lição

O erro é silencioso: o programa não trava nem lança exceção, só devolve números errados, e diferentes a cada execução. A correção não foi usar `synchronized` ou `AtomicLong`, que serializariam o acesso e destruiriam o ganho. Foi **eliminar o compartilhamento**: cada thread soma no seu acumulador e a junção acontece uma única vez, no final.

15

Como a arquitetura pode evoluir

Quando nem 8 threads bastarem: o gancho para a Unidade 2.

Threads escalam dentro de uma máquina (**escala vertical**). Quando o histórico crescer para centenas de milhões de registros, ou quando muitos analistas consultarem ao mesmo tempo, o próximo passo é **escala horizontal**: o mesmo padrão dividir → mapear → reduzir, só que entre máquinas.

1. Réplicas

Várias instâncias do serviço atrás de um **balanceador de carga**. Resolve muitas consultas simultâneas (mais analistas), mas cada consulta ainda percorre a base inteira.

2. Particionamento

Sharding do histórico por região ou estado. Cada nó calcula o acumulador parcial da sua região e um agregador soma os parciais: exatamente o que fizemos com as threads, agora com máquinas. A soma associativa de inteiros continua garantindo o resultado idêntico.

3. Agregação incremental

Parar de recalcular tudo: cada nova requisição (US01) publica um evento em uma **fila** (Kafka, RabbitMQ) e um consumidor soma na célula tipo $\times$ semana. O painel lê os agregados prontos, $O(t \cdot w)$, de um **modelo de leitura com cache** (CQRS). A consulta deixa de ser $O(n)$.

4. Processamento distribuído

Para análises pesadas e exploratórias, motores como **Apache Spark** executam o map-reduce entre dezenas de nós, com tolerância a falhas.

A ideia que atravessa tudo

O que tornou a US07 paralelizável dentro de uma JVM (dados independentes, estado local por trabalhador e uma redução associativa no final) é o mesmo princípio que a torna distribuível entre máquinas. Threads aceleram **um volume** em uma máquina; sistemas distribuídos aceleram **volumes maiores que uma máquina**.

16

Conclusão

Escolhemos a **US07, Análise Preditiva de Demanda por Tipo Sanguíneo**, porque é a operação do HemoTrack cujo gargalo, na escala nacional, é processamento: um laço $O(n)$ sobre milhões de requisições, com conversão de fuso e cálculo de semana epidemiológica por registro. Seus dados se dividem em fatias independentes e a agregação é uma soma associativa, o que permite paralelizar sem mudar a resposta.

Implementamos o serviço em Java 21 com Spring Boot, com versões sequencial, com 2, 4 e 8 threads de plataforma (`ExecutorService`) e com threads virtuais. Todas devolvem exatamente o mesmo resultado, comprovado por assinatura SHA-256 e por testes automatizados; uma versão errada de propósito mostrou a race condition que o desenho com acumuladores locais evita.

Na máquina de 16 núcleos do grupo, o endpoint com 1 milhão de registros caiu de **90,1 ms para 33,6 ms com 4 threads (2,68×)**. O ganho não foi linear, pela parte serial (Amdahl, $f \approx 16\%$), pelo custo fixo da requisição, pela memória compartilhada e pela coordenação entre threads, e a Big-O não mudou: continua $O(n)$, só dividido entre núcleos. Aqui as threads fazem **paralelismo** (acelerar um volume), diferente da **concorrência** da Mesa DJ (atender eventos simultâneos). E quando uma máquina não basta, o mesmo padrão dividir → mapear → reduzir leva a solução para um sistema distribuído.

Para o analista, o resultado é direto: **O-** tem ruptura praticamente certa no próximo trimestre e **AB-** está no limite. São esses os tipos que as campanhas de doação devem priorizar.

17

Referências

- Enunciado da atividade de paralelismo na camada de aplicação e histórias de usuário do HemoTrack (US01 a US07), Infraestrutura de Software, CESAR School, 2026.2.
- AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Spring Joint Computer Conference*, 1967.
- GOETZ, B. et al. *Java Concurrency in Practice*. Addison-Wesley, 2006. (Executores, confinamento de estado em threads, race conditions.)
- OpenJDK. *JEP 444: Virtual Threads*. Java 21, 2023.
- Oracle. Documentação da API Java 21: `java.util.concurrent.ExecutorService`, `java.time.ZoneId`.
- ABRAMOWITZ, M.; STEGUN, I. A. *Handbook of Mathematical Functions*. 1964. Fórmula 7.1.26 (aproximação da função erro).
- Spring Boot Reference Documentation, versão 3.3.
- Ministério da Saúde. Calendário de semanas epidemiológicas (a semana epidemiológica vai de domingo a sábado).